Payfonte — integration assessment and commercial read

Written: 2026-10-20 · **Sources:** docs.payfonte.com (read in full) + Payfonte's pricing reply of 2026-10-20 · **Our numbers:** `shared/domain.ts` (FEE_PCT, RAIL_SPREAD_BPS) and `server/src/core/settings.ts` (`pricing.costs`)

---

1. The one sentence that decides this

> *"The FX spread typically ranges from 4% to 7% … However, **no FX spread applies to local currencies XAF and XOF.** If you fund your account in a foreign currency, an FX spread may apply when converting the funds into the relevant local currency."*

Our entire inbound leg is BTC, USDT and USDC. Every one of those is a foreign currency to Payfonte. So the default path — send them value in crypto/USD, have them pay out XAF — lands squarely in the 4–7% band.

Against our own gross take of **4.00%** (FEE_PCT 2.5% + Lightning spread 150 bps):

FUNDING ROUTE	PAYOUT	FX	CRYPTO-IN	TOTAL COST	MARGIN
Today (assumed rail cost)	1.5%	—	0.1%	1.60%	**+2.40%**
Payfonte 1.5%, **funded in XAF**	1.5%	0%	0.1%	1.60%	**+2.40%**
Payfonte at 1.3% tier	1.3%	0%	0.1%	1.40%	**+2.60%**
Payfonte at 1.2% tier	1.2%	0%	0.1%	1.30%	**+2.70%**
Payfonte 1.5%, **funded in USD/USDT**	1.5%	4%	0.1%	5.60%	**−1.60%**
Same, at the top of their band	1.5%	7%	0.1%	8.60%	**−4.60%**

Conclusion: Payfonte is viable as a payout rail only if we fund the disbursement wallet in XAF/XOF that we source ourselves. We must never route the crypto→XAF conversion through them. At 4% FX we lose 1.6% of principal on every transfer; at 7% we lose 4.6%. No volume tier recovers that — the tiers move the payout leg by 0.2–0.3%, an order of magnitude less than the FX exposure.

This is not a reason to walk away. It is a reason to structure the deal as **payout rail only**, which is exactly what the "no FX spread on XAF and XOF" clause permits.

Where XAF funding comes from, in order of preference:

1. **Our own collections.** This is what the Interop v2 design already assumes — MoMo in funds MoMo out, and the crypto leg nets off centrally. Blocked today: our only collection rail is Peexit, which is NXDOMAIN (see `docs/peexit/`).
2. **Our own treasury conversion**, at whatever spread we can source locally — the number to beat is 4%, which is not a hard target.

2. The pricing, read closely

Rates. 1.5% launch for 90 days (Côte d'Ivoire + Cameroon payouts), then tiered on *combined MoMo›Me + Wul* monthly volume: 1.5% under $100k, 1.3% from $100k, 1.2% from $1M. Marginal, not cliff-edged — their own example charges the first $99,999 at 1.5% and the remainder at 1.3%. That is the right shape and worth confirming in writing.

1.5% is exactly our modelled payout cost (`pricing.costs.payoutPct = 0.015`). So on the assumption, this is margin-neutral, not a saving. Whether it is cheaper *in fact* depends on what Peexit actually invoices — which we record per payment as `railCostSource: "invoice"` (`server/src/core/railCosts.ts` reads the exact fee off Peexit's settled row). **Pull that before negotiating further**: Admin → Reports, and compare the invoiced percentages against 1.5%. If Peexit's real invoice is under 1.5%, the launch rate is a step backwards on Cameroon and the case rests entirely on coverage and resilience, not price.

What the tiers mean in traffic. At the live rate from production (578.7 XAF/USD) and our 5,000 XAF modal ticket:

- $100k/month ≈ 57.9M XAF ≈ **11,573 payments/month ≈ 385/day**
- $1M/month ≈ **115,731 payments/month ≈ 3,857/day**

Tiering on *combined* volume helps us reach those sooner on Wul's back. It also means our rate can rise if Wul's volume falls. They have offered prospective-only repricing, which covers us on already-processed transactions but not against an upward move next month. Ask for a floor: once a tier is reached, it holds for N months.

The XAF 25,000 per-transaction cap is irrelevant to MoMo›Me. It only beats 1.5% above **1,666,667 XAF (~$2,880)**. Our fees at the presets are 75 / 150 / 375 / 750 XAF. Do not trade anything away for that cap on MoMo›Me's behalf — it is a Wul B2B instrument. It would start to matter for merchant settlement and Treasury sweeps, not consumer remittance.

No setup fee, no settlement fee, no failed-transaction fee, no minimum commitment. All four confirmed, and all four are genuinely good. "No fee on failed transactions" matters more than it looks: our 4–9 September failure data was dominated by expired invoices, and a rail that bills for failures turns an outage into a cost.

3. GIMACPAY is the real prize, with one trap

GIMACPAY will cover **Republic of Congo, Chad, Cameroon, Gabon and Central African Republic** — that is **all five** of our declared CEMAC markets (`shared/domain.ts`: `CM` active; `GA`, `TD`, `CG`, `CF` planned). Their current provider table does *not* list CAR at all, so GIMACPAY closes precisely the gap in their own coverage.

The trap: GIMAC "supports fixed-fee pricing", and fixed fees are worse for us. At our ticket sizes, 1.5% is 75 XAF on 5,000 and 150 XAF on 10,000. A fixed fee only helps if it is below those — and a fixed remittance fee below 75 XAF is implausible. So for consumer remittance, **percentage pricing beats fixed**, and a GIMAC migration could *raise* our unit cost even as it is presented as a reduction.

Ask: what is the GIMACPAY fixed fee in XAF, and can we keep percentage pricing on small-ticket consumer flows while Wul takes the fixed-fee cap on large B2B? Those are opposite optimisations and should not be forced onto one schedule.

4. How it integrates (no core changes needed)

Three of our registry interfaces take it as a drop-in:

OUR SEAM	PAYFONTE
`PayoutAdapter.disburse()`	`POST /billing/v1/disbursements`
`PayoutAdapter.queryStatus()`	`GET /billing/v1/disbursements/verify/{reference}`
`PayoutAdapter.balance()`	wallet balance by `country` + `currency`
`PayoutAdapter.verifyCallback()`	`x-webhook-signature` : HMAC-SHA512 over the raw body, keyed with `client-secret`
`PayoutAdapter.parseCallback()`	`disbursement.status` → `processing` / `success` / `failed`
`PayoutAdapter.payoutFeePct?()`	no fee endpoint → `null` ; the contract goes in `settings.pricing.contracts` instead
`CollectAdapter.collect()`	`POST /payments/v1/payments/direct-charge` (server-to-server, USSD/STK push)
identity `resolve()`	`POST /billing/v1/transfer-recipients/validate` — **but see below**

Adding it as a third payout rail needs no routing changes: `priority` + balance-aware selection + the shared `railHealth` failover already handle it, and the cost model will prefer the cheaper funded rail on its own.

Two things it does **not** do:

- It does not touch the crypto leg. IBEX and phoenixd are unaffected.
- **It does not restore recipient verification.** The documented response of `transfer-recipients/validate` has no name field — only `accountLabel`, a display string whose example happens to contain a name. Our provider contract already refuses to invent a holder's name (`200 without accountName → UNKNOWN`). Parsing a name out of a label the vendor does not contract is how a sender gets shown the wrong person. Treat it as a reachability check like pawapay until they confirm a structured field.

5. Open questions — must be answered before live money

1. **XAF amount units.** Their amount spec states one rule for every currency, `amount = major × 100` , and shows `10.00 XOF -> 1000` . **XOF and XAF are zero-decimal currencies**; XAF is not mentioned on the page at all. Does `amount: 5000` mean 5,000 XAF or 50 XAF? This is a 100× error in either direction and it is the single highest-risk unknown. Get it in writing, then pin it with a test.
2. **Idempotency.** `reference` "must be unique for every transaction" is all the docs say. What does a retried POST with an existing reference return — the original transaction, or a duplicate payout? Our `disburse()` contract is idempotent on `idempotencyKey` and a network timeout plus "must be unique" is how a double payout happens.
3. **Terminal statuses.** The webhook guide says to finalise "only for terminal statuses" and never lists them. We will keep `queryStatus` as the settlement gate regardless, but we need the list.
4. **Does `transfer-recipients/validate` return a structured account-holder name?** If yes, it replaces `peexit_verify` and recipient verification comes back. If no, it does not.

5. **Regulatory standing.** They cite PCI DSS and GDPR; no central-bank licence or authorisation is mentioned anywhere in the docs. Against COBAC / BEAC Instr. 002/2026 and ANIF obligations, we need their licensing position per market in writing (`compliance@payfonte.com`).
6. **Tier floor.** Once a tier is reached, does it hold? Prospective-only repricing protects history, not next month.
7. **GIMACPAY fixed fee**, in XAF, and whether percentage pricing can be retained for small-ticket consumer flows.

6. Verdict

Pursue it — as a **payout rail funded in XAF, and a second collection rail**, under the orchestration model where we keep our own operator rates. That solves a live single point of failure: today one vendor's missing DNS record leaves payouts on one rail and collections on none.

Do not take the aggregation model for Cameroon. Their own `transfer-recipients/validate` schema carries a `"Pawapay mobile money"` account shape, which indicates they route some corridors through pawapay — a rail we already hold directly. Paying a layer to reach it is a cost increase.

And do not let them do the FX.
